

Learning Objectives

Learners will be able to...

- Describe the uses for and implement basic frequency analysis
- Create a Bag of Words model

info

Make Sure to Know

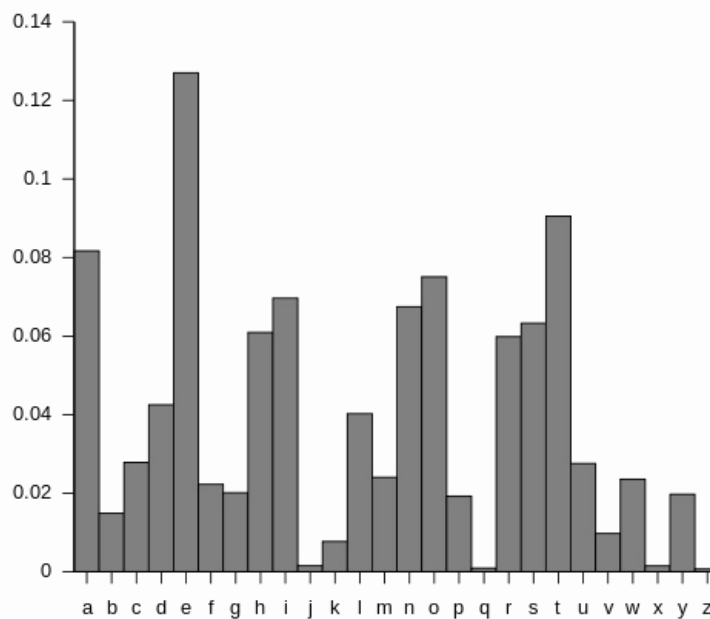
Intermediate Python.

Limitations

In this assignment, we only work with the NLTK library.

History of Letter Frequency Analysis

Languages have different frequencies of the usage of letters. For example, the frequencies for the English language are below:



As you can see above, the most commonly used letter is “E”. The nonsense phrase “ETAOIN SHRDLU” is a way to remember the order of the twelve most frequent letters.

There are a number of websites that will count the letters for you. [Try this one out](#) by typing into the “Text to Analyze” box and pressing “Launch Analysis”.

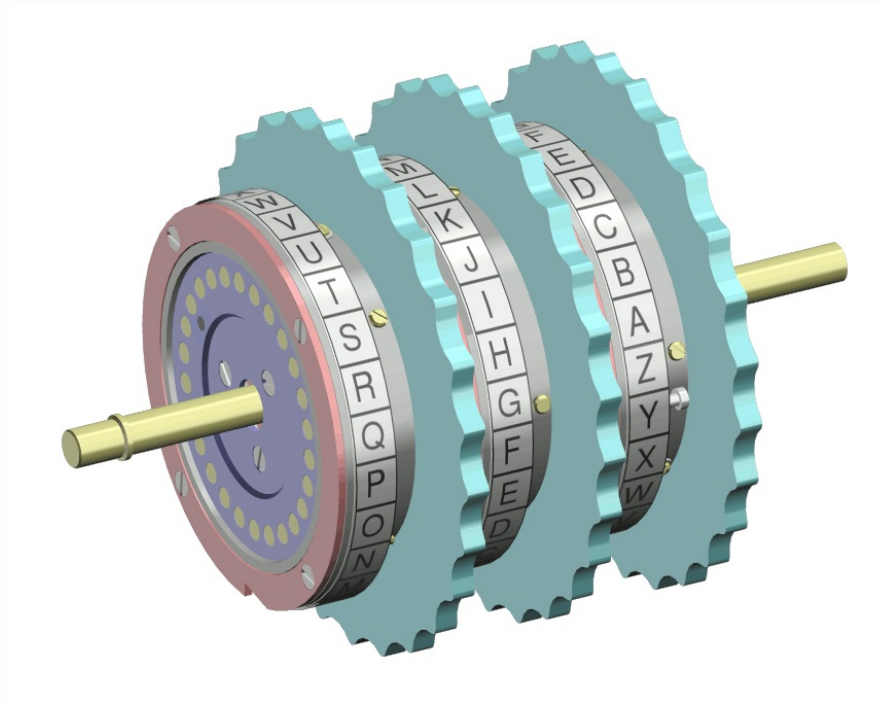
History of Letter Frequency Analysis

Letter frequency analysis has been known about since at least 900 AD in *A Manuscript on Deciphering Cryptographic Messages* which was published in Arabic and possibly used the Qur’an to develop the letter frequency. By the 1400s the concept was published about in both Italian and Latin.

In World War II, several of the Axis powers ciphers were breakable by frequency analysis which was done by hand, so British and American forces recruited codebreakers using crossword puzzles. As computers

became available to perform these analyses, classical ciphers which can be broken using frequency analysis became obsolete.

Rotor machines (pictured below) such as [the Enigma machine](#) used in World War II replaced these simple ciphers as they could not be broken by the same simple frequency analysis.



Counting Letters with Python

On the left is a method called `getFrequency` which returns a count of the letters in `message`.

The are in the `main` method at the end of the file.

Click the button below to see the letter frequency of `message`:

challenge

Try these variations:

- Change the method call to `getFrequencyOrder`:

```
python    print(getFrequencyOrder(message))
```
- Change the text in `message` to see if the most frequently used letters stay the same

Word Frequency

Similar to how letter frequency analysis proved useful in cryptography, word frequencies were found to be useful in Natural Language Processing.

What are the most common words in English?

Each language has a unique distribution of words. Here are the top 10 most frequently used words in English:

Word	Count
the	23135851162
of	13151942776
and	12997637966
to	12136980858
a	9081174698
in	8469404971
for	5933321709
is	4705743816
on	3750423199
that	3400031103

You can see the full list of the most frequently used words in English here: <https://www.kaggle.com/datasets/rtatman/english-word-frequency>

The top words may look familiar – many of them are **stop words** or words that we often filter out because they hold little meaning.

Now, try filtering the stop words out of the top ten words by running the code on the top-left.

challenge

Try these variations:

- Try the top 30 words:

```
python      most_common_words = """the of and to a in for is
on that    by this with i you it not or be are      from at
as your all have new more an was"""
```

- Try the top 50 words:

```
python      most_common_words = """the of and to a in for is
on that    by this with i you it not or be are      from at
as your all have new more an was      we will home can us
about if page my has      search free but our one other do no
information time"""
```

Counting Words with Python

NLTK has a helpful FreqDist or Frequency Distribution utility.

Replace the code in the top-left pane with the following:

```
from nltk.corpus import webtext
from nltk.probability import FreqDist
from nltk.tokenize import word_tokenize

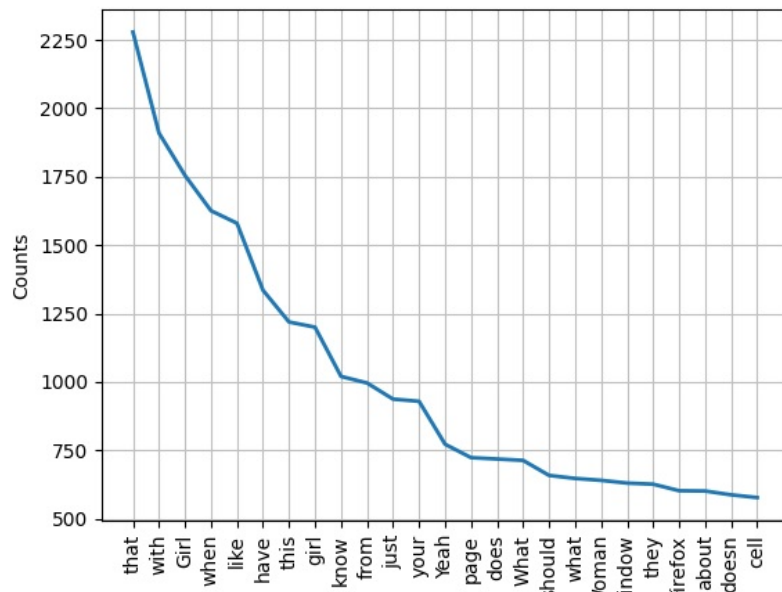
wt_words = webtext.words()
data_analysis = FreqDist(wt_words)

# Let's take the specific words only if their frequency is
greater than 3.
filter_words = dict([(m, n) for m, n in data_analysis.items() if
    len(m) > 3])
data_analysis = FreqDist(filter_words)

data_analysis.plot(25, cumulative=False)
```

Run the code and watch for a plot in the bottom-left pane:

You should see something that looks like:



Notice the **25** in the `plot` command – which plots only the top 25 most frequent words. You can easily print a list of those words using the `most_common` function:

```
most_common = data_analysis.most_common(25)
print("\nIn Order of Most Common Frequencies: ")
for word in most_common:
    print(word)
```

Note: Comment out the line `data_analysis.plot(25, cumulative=False)` to ensure you see the printed output.

challenge

Try these variations:

- Print just the most common word using the `max()` function:

```
python    print("\nMost common word: ")
print(frequency_analysis.max())
```
- Try printing words which show up more than a certain threshold:

```
python    most_common = frequency_analysis.most_common()
print("\nIn Order of Most Common Frequencies: ")    for word
in most_common:    if word[1] > 1000:
print(word)
```
- Try printing the **hapaxes** or the words that show up exactly once:

```
python    hapaxes = frequency_analysis.hapaxes()
print("\nHapaxes or Words occuring only once: ")    for word
in hapaxes:    print(word)
```

What is a Bag of Words?

While a list of most frequent words is helpful, when humans try to make sense of text we often make word clouds. In word clouds, repeating words (or words with high frequency) get larger or a more eye-catching color.

Here is an example of a word cloud about trees:



Many algorithms, particularly in Machine Learning, struggle with text too. To simplify natural language for them, we use an idea similar to word clouds: a bag of words.

Similar to a word cloud, a bag of words:

- * ignores the order words appear in the text
- * is represented as a set of the unique words in the text
- * keeps track of the count or frequency of each word

Bag of Words Example

The canonical example for a bag of words model consists of 2 **documents**.

Document 1:

John likes to watch movies. Mary likes movies too.

Document 2:

Mary also likes to watch football games.

These documents are pre-processed or tokenized to create the following lists:

```
["John", "likes", "to", "watch", "movies", "Mary", "likes", "movies", "too"]  
["Mary", "also", "likes", "to", "watch", "football", "games"]
```

The unique words are then identified and combined to create the **vocabulary**:

```
["John", "likes", "to", "watch", "movies", "Mary", "too", "also", "football", "games"]
```

We can then create a vector with a count for each word's appearance.

For example, for

> John likes to watch movies. Mary likes movies too.

We get the following vector:

```
[1, 2, 1, 1, 2, 1, 1, 0, 0, 0]
```

For example, for

> Mary also likes to watch football games.

We get the following vector:

```
[0, 1, 1, 1, 1, 0, 1, 0, 1, 1]
```

Implement a Bag of Words

The start of a Bag of Words implementation is on the left using concepts we have already covered. Click on the links below to highlight the section of code they are describing:

*

* This code could be easily re-configured to contain file names or corpora

*

*

There are a couple of big design decisions when implementing a bag of words model.

Design Decision 1: Defining the vocabulary of known words (or tokens)

The first design decision is what is included in the vocabulary. At this point, in `unique_words` we have every word that appears in the document set.

There are a number of things you could choose to do:

* remove stop words

* remove punctuation

* ignore case

* do stemming or other lemmatization to ignore tense/plural

For our example, let's start by just removing stop words:

```
## Process out stop words to create VOCABULARY
stop_words = set(stopwords.words('english'))
vocabulary = [element for element in unique_words if element not
               in stop_words]

print(vocabulary)
```

challenge

Try these variations:

- Try removing punctuation from the vocabulary:

```
``python
## Process out stop words and punctuation to create VOCABULARY
stop_words = set(stopwords.words('english'))
unique_words = filter(lambda word: word not in
string.punctuation, unique_words)
vocabulary = [element for element in unique_words if element not
in stop_words]

print(vocabulary)
``
```
- Try forcing all words to lower-case to ignore case:

```
python    ## remove duplicates so only UNIQUE words remain
all_words = [word.lower() for word in all_words] ## ADDED
unique_words = sorted(list(set(all_words)))
```
- Try using `.casefold()` to ignore case as part of stop-word filtering:

```
python    vocabulary = [element for element in unique_words
if element.casefold() not in stop_words]
```

Up to now, we have assumed the vocabulary is a set of single words – but that is not necessarily the case! A bag of words vocabulary can contain n-grams, or substrings made up of n words, instead of single words. This helps preserve some sense of the order of the words in a bag of words model.

info

Large Vocabulary? Hashing!

Depending on your design decisions and the size of your initial document set, you could be looking at a large vocabulary. While we can sort the vocabulary and use something like binary search to help speed things up, an even better approach is hashing which allows you to access a specific vocabulary entry in constant time.

Design Decision 2: How to score the presence of known words

In the example on the previous page, we simply counted the number of appearances of a word. Let's start by implementing that scoring system:

```
## Generate bag of word vectors by counting instances of each word
for document in documents:
    words = word_tokenize(document)
    bag_vector = np.zeros(len(vocabulary))
    for w in words:
        for i,word in enumerate(vocabulary):
            if word.lower() == w.lower():
                bag_vector[i] += 1

print("{0}
\n{1}\n".format(document,np.array(bag_vector)))
```

If you have ignored case, your output should look similar to:

```
['also', 'football', 'games', 'john', 'likes', 'mary', 'movies',
'watch']

John likes to watch movies. Mary likes Movies too.
[0. 0. 0. 1. 2. 1. 2. 1.]

Mary also likes to watch football games.
[1. 1. 1. 0. 1. 1. 0. 1.]
```

There are other scoring systems aside from count. The vector could consist of only 0's and 1's indicating whether a word was present or not (ignoring duplicate entries).

An interesting variation on count is the idea of normalized frequency where the count for a particular word in the vocabulary is divided by the total word count:

$$frequency = \frac{count_{word}}{count_{total}}$$

Notice that a number of design decisions have been made for us by `CountVectorizer`. For example:

- 1. All words are forced to lower case (`lowercase=True`)
- 1. The vocabulary is full of single words (`analyzer='word'`)
- 1. Tokens are made up of 2 or more alphanumeric characters (punctuation is completely ignored)
- 1. Scoring against the vocabulary is done by count (`binary=False`)
- 1. Stop words are not filtered out (`stop_words=None`)

These assumptions about design decisions can be overridden.

challenge

Try these variations:

- Try removing stop words:

```
python    vectorizer = CountVectorizer(stop_words='english')
```
- Try scoring based on existence instead of counting:

```
python    vectorizer = CountVectorizer(binary=True)
```

N-grams example

Let's create a bag of words model with 2-grams:

```
vectorizer = CountVectorizer(analyzer='word', ngram_range=(2,  
2))  
X = vectorizer.fit_transform(documents)
```

We can then print out the vocabulary to see the 2-grams:

```
print(vectorizer.get_feature_names_out())
```

We can also print out the vectors to see the usage:

```
print(X.toarray())
```

Bag of Words Limitations

First, bag of words models can often yield vectors that are mostly zeros. These are called **sparse** because relative to the total size of the vector, there are few non-zero entries.

Sparse vectors are harder to work with in terms of computational resources (space and time complexity) and informationally (it is hard to make sense with so little information in such a large vocabulary).

Vocabularies require careful design in an attempt to decrease their overall size – decreasing the sparsity.

Second, discarding word order, even with the use of n-grams, causes context-loss. This can obscure the meaning of words as they appear in the document (i.e. semantics). Consider how the bag of words would handle the same words differently arranged (“this is interesting” vs “is this interesting”) or the use of synonyms (“old car” vs “used car”).

Formative Assessment 1

Formative Assessment 2